6.OpenClaw Tools Introduction

Tools are the "capability interfaces" exposed by OpenClaw to agents, enabling models to complete real operations in a safer and more structured way, such as reading/writing files, running commands, managing sessions, web access, UI automation, etc. Compared to the old `openclaw-*` Skills, tools are typed, controllable, and can be precisely limited through configuration to define "what they are allowed to do / not allowed to do".

1. Key Concepts

1.1 Relationship between Tools and Skills

- Tools: System-level "capability interfaces", typed, configurable, and can be restricted by provider/model
- Skills: More focused on "reusable workflow/plugin capabilities", many old skills will be replaced or weakened by the tool system

If you want agents to "do things", prioritize tools; if you want to encapsulate a process into reusable capabilities, then consider Skills.

1.2 More Tools Isn't Always Better

The more tools available, the more the model can do, but the greater the risk. Best practice: by default, only enable tools you really need, and separately disable or on-demand enable high-risk capabilities (especially running commands).

1.3 Core of Tool Strategy: profile + allow/deny

Tool sets are typically trimmed in this order:

1. `tools.profile` first provides a "basic allowed set"
2. If `tools.byProvider` is configured, it will further tighten for specified provider or provider/model (it can only shrink the tool set)
3. Finally use `tools.allow` / `tools.deny` to precisely tighten or loosen (`deny` takes priority)

2. Quick Start

OpenClaw's tool permission policies are written in `openclaw.json`. Generally, we don't need to configure manually

- `deny` has higher priority than `allow`
- Supports `*` wildcards, and matching is case-insensitive

2.1 Choose a Tool Configuration Profile

Common profiles (from "least to most capability"):

- `minimal`: Only `session_status`
- `messaging`: Focused on messaging capabilities + sessions capabilities
- `coding`: Focused on development/engineering capabilities (files, runtime, sessions, memory, images, etc.)
- `full`: No restrictions (same as not setting)

Example: Enable coding profile

```
{
  "tools": {
    "profile": "coding"
  }
}
```

2.2 Precise Control with allow/deny (Recommended to Start with deny)

Example: Globally disable `browser` tool

```
{
  "tools": {
    "deny": ["browser"]
  }
}
```

Example: Based on coding, disable all runtime tools (exec / bash / process)

```
{
  "tools": {
    "profile": "coding",
    "deny": ["group:runtime"]
  }
}
```

2.3 Further Tighten for Specific Provider/Model (byProvider)

You can keep global defaults unchanged and only further tighten the tool set for a specific provider or provider/model (it can only shrink, not expand).

Example: Use coding globally, but Google Antigravity uses minimal

```
{
  "tools": {
    "profile": "coding",
    "byProvider": {
      "google-antigravity": { "profile": "minimal" }
    }
  }
}
```

Example: Further reduce tools for `openai/gpt-5.2`

```
{
  "tools": {
    "allow": ["group:fs", "group:runtime", "sessions_list"],
    "byProvider": {
      "openai/gpt-5.2": { "allow": ["group:fs", "sessions_list"] }
    }
  }
}
```

3. Tool Groups (group:*)

In `tools.allow` / `tools.deny`, you can use `group:*` to control a group of tools at once:

Group Name	Expands To
`group:runtime`	exec、bash、process
`group:fs`	read、write、edit、apply_patch
`group:sessions`	sessions_list、sessions_history、sessions_send、sessions_spawn、session_status
`group:memory`	memory_search、memory_get
`group:web`	web_search、web_fetch
`group:ui`	browser、canvas
`group:automation`	cron、gateway
`group:messaging`	message
`group:nodes`	nodes
`group:openclaw`	All built-in OpenClaw tools (excluding provider plugins)

Example: Only allow file tools + browser

```
{
  "tools": {
    "allow": ["group:fs", "browser"]
  }
}
```

4. High-Risk Tools: How to Configure Safely

4.1 Command Execution Related (group:runtime)

`group:runtime` contains execution capabilities (e.g., exec / bash / process). For these capabilities, it's recommended to follow:

- Default disabled: First `deny: ["group:runtime"]`, then gradually enable when needed
- Only enable for "specific agents": Concentrate high-risk capabilities on dedicated agents
- Only enable for "specific provider/model": Enable more tools for stable/trusted models

4.2 Tips for apply_patch

`apply_patch` belongs to file editing tools; in some versions it's an experimental feature and may require additional switches to enable (e.g., through `tools.exec.applyPatch.enabled`, and only supported by some models). If you find that the model can edit files but cannot call apply_patch, prioritize checking tool policies and related switches.

4.3 Isolate Tools by Agent (Recommended Practice)

You can set a global tool policy and then separately override for specific agents, concentrating "high-privilege capabilities" on a few agents.

Example: Use coding globally, but support agent only uses messaging, with additional Slack tool allowed

```
{
  "tools": { "profile": "coding" },
  "agents": {
    "list": [
      {
        "id": "support",
        "tools": { "profile": "messaging", "allow": ["slack"] }
      }
    ]
  }
}
```

5. Recommended Configuration Templates (Ready to Use)

5.1 Daily Conversations (More Secure)

Goal: Only chat + view session status, without giving it the ability to "mess with the system".

```
{
  "tools": {
    "profile": "minimal"
  }
}
```

5.2 Learning/Documentation (Allow web search, but not command execution)

```
{
  "tools": {
    "profile": "coding",
    "deny": ["group:runtime"],
    "allow": ["group:web"]
  }
}
```

5.3 Development/Debugging (Can read/write files, but treat command execution as "optional")

```
{
  "tools": {
    "profile": "coding",
    "deny": ["group:runtime"]
  }
}
```

When you actually need to execute commands, temporarily enable `group:runtime`, then disable it again when done.

6. Common Questions

6.1 Configured allow but Not Taking Effect

- Check if `deny` also hits the same tool (deny has priority)
- Check if the tool name is spelled correctly (matching is case-insensitive, but the name must exist)
- If `tools.allow` only contains unknown/unloaded plugin tools, the system may ignore the allow list to ensure core tools are available

6.2 One Model Can Use Tools, Another Model Cannot

- You may have set `tools.byProvider` (provider or provider/model) that imposes additional restrictions on that model
- Some provider endpoints have inconsistent tool support, it's recommended to use smaller tool sets for unstable endpoints

6.3 Which Profile Should I Choose?

- Just want to chat: `minimal`
- Daily messaging + session management: `messaging`
- File/engineering tasks: `coding`
- You clearly know what you're doing and can bear risks, want complete and powerful OpenClaw: `full`

7. Official Documentation

- OpenClaw Tools Documentation: https://docs.openclaw.ai/tools